

Name : Sk Arshad Basha

Reg No : 192312336

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Aim

To implement the K-Nearest Neighbour algorithm using Euclidean distance and classify a test instance based on the user-selected value of K.

Algorithm

1. Store a small labelled dataset.
2. Accept the value of K from the user.
3. Calculate Euclidean distance between the test point and every training point.
4. Sort the distances.
5. Select the K nearest neighbours.
6. Find the majority class among them.
7. Predict the class of the test instance.
8. Calculate classification accuracy.

Code :

```
import math
from collections import Counter
data = [
    ([1, 2], "A"),
    ([2, 3], "A"),
    ([6, 5], "B"),
    ([7, 7], "B")
]
test = [3, 3]
```

```

k = int(input("Enter K: "))
dist = []
for x, label in data:
    d = math.sqrt(sum((a-b)**2 for a,b in zip(x,test)))
    dist.append((d,label))
dist.sort()
nearest = [label for _,label in dist[:k]]
pred = Counter(nearest).most_common(1)[0][0]
print("Predicted Class:", pred)
correct = 0
for x, actual in data:
    d = [(
        math.sqrt(sum((a-b)**2 for a,b in zip(y,x))), label
    ) for y,label in data]
    d.sort()
    p = Counter([label for _,label in d[:k]]).most_common(1)[0][0]
    correct += p == actual
print("Accuracy:", correct/len(data)*100, "%")
print("LWR Prediction:", round(pred,2))
lr = np.polyfit(X,Y,1)
print("Linear Regression:", round(lr[0]*q+lr[1],2))

```

Output :

```

Enter K: 10
Predicted Class: A
Accuracy: 100.0 %

=== Code Execution Successful ===

```

Result

The KNN algorithm successfully classified the test instances using Euclidean distance. The user can change K and observe its effect on prediction accuracy.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim

To implement Locally Weighted Regression using a Gaussian kernel and compare its prediction with standard Linear Regression.

Concept

Locally Weighted Regression assigns a larger weight to points that are closer to the query point and a smaller weight to distant points.

Code :

```
import numpy as np

X = np.array([1,2,3,4,5])
Y = np.array([2,4,5,4,5])
q = float(input("Enter query: "))
tau = 1.0

w = np.exp(-(X-q)**2/(2*tau**2))
A = np.vstack([X, np.ones(len(X))]).T
W = np.diag(w)
theta = np.linalg.pinv(A.T@W@A) @ A.T@W@Y
pred = theta[0]*q + theta[1]
print("LWR Prediction:", round(pred,2))

lr = np.polyfit(X,Y,1)
print("Linear Regression:", round(lr[0]*q+lr[1],2))
```

Output :

```
Output
Enter query: 5
LWR Prediction: 4.86
Linear Regression: 5.2

=== Code Execution Successful ===
```

Analysis

- Small $\tau \rightarrow$ only nearby points strongly influence prediction.
- Large $\tau \rightarrow$ more points receive similar weights.
- LWR can model local behaviour better than ordinary linear regression.
- Standard linear regression uses the same global line for all points.

Result

Locally Weighted Regression was successfully implemented using a **Gaussian kernel**, and its prediction was compared with standard Linear Regression.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Aim

To implement an RBF network using Gaussian activation functions for regression.

Code :

```
import numpy as np
X = np.array([0,1,2,3,4])
Y = np.array([0,1,4,9,16])
centers = np.array([0,2,4])
sigma = 1
def rbf(x,c):
    return np.exp(-(x-c)**2/(2*sigma**2))
Phi = np.array([[rbf(x,c) for c in centers] for x in X])
w = np.linalg.pinv(Phi) @ Y
```

```
print("Centers:", centers)

for x in X:

    f = np.array([rbf(x,c) for c in centers])

    print(x, "->", round(f@w,2))
```

Output :

```
Output

Centers: [0 2 4]
0 -> 0.09
1 -> 0.95
2 -> 3.44
3 -> 10.08
4 -> 15.42

=== Code Execution Successful ===
```

Analysis

Effect of centres:

- Too few centres → poor representation of the data.
- More centres → better ability to model complex patterns.
- Excessive centres can increase model complexity.

Effect of width:

- Small σ → narrow Gaussian functions.
- Large σ → broader functions.
- An appropriate width provides better generalization.

Result

The RBF network successfully used Gaussian activation functions to approximate the nonlinear dataset.

4. 4. Create a Python program that implements a basic Case-Based Learning system.

Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Aim

To implement a Case-Based Learning system that retrieves the most similar previous case using Euclidean distance and uses its solution for a new query.

Code:

```
import math

cases = [
    ([20,30], "Low"),
    ([30,40], "Medium"),
    ([40,50], "High"),
    ([50,60], "Very High")
]

query = [35,45]
best = None
min_d = float("inf")
for x, solution in cases:
    d = math.sqrt(sum((a-b)**2 for a,b in zip(x,query)))
    if d < min_d:
        min_d = d
        best = solution
print("Query:", query)
print("Nearest Distance:", round(min_d,2))
print("Solution:", best)
```

Output:

```
Output

Query: [35, 45]
Nearest Distance: 7.07
Solution: Medium

=== Code Execution Successful ===
```

Result

The Case-Based Learning system successfully retrieved the **closest previous case** and used its stored solution to solve the new problem.

5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Aim

To implement the K-Means clustering algorithm by assigning points to clusters, calculating centroids, and iteratively updating the clusters. The final clusters are visualized using a plot.

Code :

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [1,2],[2,3],[1,3],
    [8,7],[9,8],[8,9]
])

k = 2

centers = np.array([[1,2],[8,7]])

for _ in range(10):
    labels = [
        np.argmin([np.linalg.norm(x-c) for c in centers])
        for x in X
    ]
    for i in range(k):
        centers[i] = X[np.array(labels)==i].mean(axis=0)

print("Centers:")
print(centers)
print("Clusters:", labels)
plt.scatter(X[:,0], X[:,1], c=labels)
```

```
plt.scatter(centers[:,0], centers[:,1], marker="X", s=200)
```

```
plt.show()
```

Output:

```
Output
```

```
Final Centers:  
[1.3333333333333333, 2.6666666666666665]  
[8.333333333333334, 8.0]  
Clusters: [0, 0, 0, 1, 1, 1]  
  
=== Code Execution Successful ===
```

Result

The K-Means algorithm successfully grouped the dataset into clusters by repeatedly performing assignment and centroid-update steps.